blocks/blocks_and_bindings.rb
```
def my_method
  x = "Goodbye"
  yield("cruel")
end

x = "Hello"
my_method {|y| "#{x}, #{y} world" } # => "Hello, cruel world"
```

When you create the block, you capture the local bindings, such as `x`. Then you pass the block to a method that has its own separate set of bindings. In the previous example, those bindings also include a variable named `x`. Still, the code in the block sees the `x` that was around when the block was defined, not the method's `x`, which is not visible at all in the block.

You can also define additional bindings inside the block, but they disappear after the block ends:

blocks/block_local_vars_failure.rb
```
def just_yield
  yield
end

top_level_variable = 1

just_yield do
  top_level_variable += 1
  local_to_block = 1
end

top_level_variable  # => 2
local_to_block      # => Error!
```

Because of the properties above, a computer scientist would say that a block is a *closure*. For the rest of us, this means a block captures the local bindings and carries them along with it.

So, how do you use closures in practice? To understand that, take a closer look at the place where all the bindings reside—the *scope*. Here you'll learn to identify the spots where a program changes scope, and you'll encounter a particular problem with changing scopes that can be solved with closures.

Scope

Imagine being a little debugger making your way through a Ruby program. You jump from statement to statement until you finally hit a breakpoint. Now catch your breath and look around. See the scenery around you? That's your *scope*.